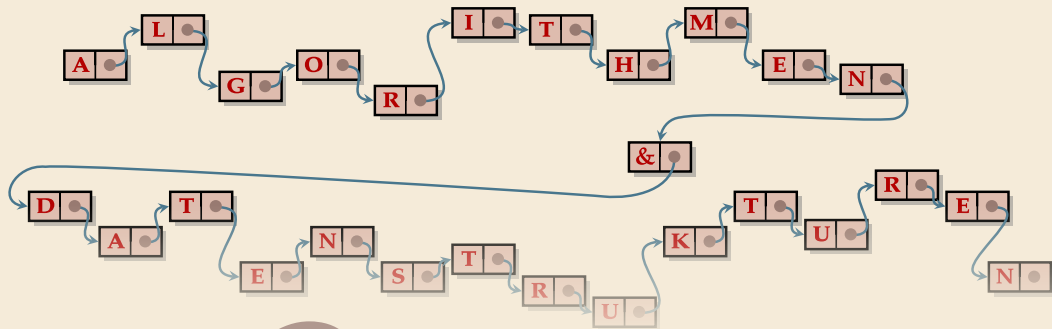




8

Dynamische Listen

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

8 Dynamische Listen

- 8.1 Abstract Data Types
- 8.2 Linear Listen
- 8.3 Stacks & Queues
- 8.4 Array-basierte Listen
- 8.5 Verkettete Listen
- 8.6 Doubling Arrays
- 8.7 Java Collections Framework

Dynamische Datenstrukturen

Bisher: im Wesentlichen alles Arrays!

- ▶ Sortieren
- ▶ Suchen
- ▶ Subsequence Sums
- ▶ 2D-Felder

außer Union-Find!



Dynamische Datenstrukturen

Bisher: im Wesentlichen alles Arrays!

- ▶ Sortieren  außer Union-Find!
- ▶ Suchen
- ▶ Subsequence Sums
- ▶ 2D-Felder

Jetzt: Dynamische Strukturen!

↪ Hier: Sequenzen von Elementen („Listen“)

8.1 Abstract Data Types

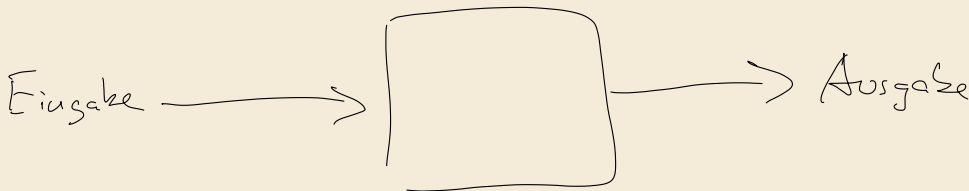
Abstract Data Types

Für algorithmische Probleme, starten mit Gegeben-Ziel-Spezifikation

► Sortieren

Gegeben: Array $A[0..n)$ von n Objekten, Sortierkriterium

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.



Abstract Data Types

Für algorithmische Probleme, starten mit Gegeben-Ziel-Spezifikation

► Sortieren

Gegeben: Array $A[0..n)$ von n Objekten, Sortierkriterium

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Abstrakter Datentyp (ADT) ist das Pendant für Datenstrukturaufgaben

- spezifiziert Operationen (Queries, Updates) auf Daten mit Semantik
- Bei Queries: Angabe welcher Rückgabewert produziert werden soll
- Bei Updates: Angabe, wie sich der Inhalt der Datenstruktur semantisch ändert

Abstract Data Types

Für algorithmische Probleme, starten mit Gegeben-Ziel-Spezifikation

► Sortieren

Gegeben: Array $A[0..n)$ von n Objekten, Sortierkriterium

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Abstrakter Datentyp (ADT) ist das Pendant für Datenstrukturaufgaben

- spezifiziert Operationen (Queries, Updates) auf Daten mit Semantik
- Bei Queries: Angabe welcher Rückgabewert produziert werden soll
- Bei Updates: Angabe, wie sich der Inhalt der Datenstruktur semantisch ändert

≈ Java Interface

≠ Implementierung/Datenstruktur: ADT gibt nur an *was* passieren soll, nicht *wie*

Beispiel *Dynamische Dekremente*

ADT *Union-Find* (Disjunkte Mengen)

- ▶ Operationen: $\text{construct}(n)$, $\text{union}(p, q)$, $\text{find}(p)$

Beispiel

ADT *Union-Find* (Disjunkte Mengen)

- ▶ Operationen: $\text{construct}(n), \text{union}(p, q), \text{find}(p)$
- ▶ Semantik
 - ▶ construct gibt Größe n des Universums fest vor (unveränderlich)
 - ▶ Zu jedem Zeitpunkt haben wir eine Partition $S_0, \dots, S_{k-1}$ von $\{0, \dots, n-1\}$ für ein k .

Beispiel

ADT *Union-Find* (Disjunkte Mengen)

- ▶ Operationen: $\text{construct}(n), \text{union}(p, q), \text{find}(p)$
- ▶ Semantik
 - ▶ construct gibt Größe n des Universums fest vor (unveränderlich)
 - ▶ Zu jedem Zeitpunkt haben wir eine Partition $S_0, \dots, S_{k-1}$ von $\{0, \dots, n-1\}$ für ein k .
 - ▶ Initial ist $k = n$ und $S_i = \{i\}$

Beispiel

ADT *Union-Find* (Disjunkte Mengen)

- ▶ Operationen: $\text{construct}(n)$, $\text{union}(p, q)$, $\text{find}(p)$
- ▶ Semantik
 - ▶ construct gibt Größe n des Universums fest vor (unveränderlich)
 - ▶ Zu jedem Zeitpunkt haben wir eine Partition $S_0, \dots, S_{k-1}$ von $\{0, \dots, n-1\}$ für ein k .
 - ▶ Initial ist $k = n$ und $S_i = \{i\}$
 - ▶ Query $\text{find}(p)$ gibt das eindeutige i zurück mit $p \in S_i$
 - ▶ Update $\text{union}(p, q)$: mit $i = \text{find}(p)$ und $j = \text{find}(q)$ ersetze S_i und S_j durch $S_i \cup S_j$.

Beispiel

ADT *Union-Find* (Disjunkte Mengen)

- ▶ Operationen: construct(n), $\text{union}(p, q)$, $\text{find}(p)$
- ▶ Semantik
 - ▶ construct gibt Größe n des Universums fest vor (unveränderlich)
 - ▶ Zu jedem Zeitpunkt haben wir eine Partition $S_0, \dots, S_{k-1}$ von $\{0, \dots, n-1\}$ für ein k .
 - ▶ Initial ist $k = n$ und $S_i = \{i\}$
 - ▶ Query $\text{find}(p)$ gibt das eindeutige i zurück mit $p \in S_i$
 - ▶ Update $\text{union}(p, q)$: mit $i = \text{find}(p)$ und $j = \text{find}(q)$ ersetze S_i und S_j durch $S_i \cup S_j$.
- ▶ Java Interface

```
1 public interface UnionFind {  
2     // UnionFind(int n);  
3     // (Konstruktoren können im interface angegeben werden.)  
4  
5     /** gibt das eindeutige i zurück mit  $p \in S_i$  */  
6     int find(int p);  
7  
8     /** ersetze  $S_i$  und  $S_j$  durch  $S_i \cup S_j$  für  $i = \text{find}(p)$ ,  $j = \text{find}(q)$  */  
9     void union(int p, int q);  
10 }
```

Clicker Question



Welche der folgenden Klassen haben Sie schon verwendet?

- A** `java.util.List`
- B** `java.util.ArrayList`
- C** `java.util.LinkedList`
- D** `java.util.Vector`
- E** `java.util.HashMap`
- F** keine davon



→ *sli.do/cs210*

Java Generics & Collections

Weit-verbreitete Datenstrukturen: **Container / Collections**

- ▶ bestehen aus einer Sammlung von Objekten

Java Generics & Collections

Weit-verbreitete Datenstrukturen: **Container / Collections**

- ▶ bestehen aus einer Sammlung von Objekten
- ▶ Objekte für Container eine "black box"  egal was drin steckt

Java Generics & Collections

Weit-verbreitete Datenstrukturen: **Container / Collections**

- ▶ bestehen aus einer Sammlung von Objekten
- ▶ Objekte für Container eine "black box" $\rightsquigarrow$ egal was drin steckt
- ▶ typische Anwendung: erlauben nur „sortenreine“ Elemente
 - ▶ Liste von Strings
 - ▶ Menge von Dateien
 - ▶ Matrix von ganzen Zahlen

$\rightsquigarrow$ möchten „sortenrein“ spezifizieren können, ohne vorher „Sorte“ festzulegen

Analog zu `int[]`, `String[]`, etc.!

Java Generics & Collections

Weit-verbreitete Datenstrukturen: **Container / Collections**

- ▶ bestehen aus einer Sammlung von Objekten
- ▶ Objekte für Container eine "black box" $\rightsquigarrow$ egal was drin steckt
- ▶ typische Anwendung: erlauben nur „sortenreine“ Elemente
 - ▶ Liste von Strings
 - ▶ Menge von Dateien
 - ▶ Matrix von ganzen Zahlen

$\rightsquigarrow$ möchten „sortenrein“ spezifizieren können, ohne vorher „Sorte“ festzulegen

Analog zu `int[]`, `String[]`, etc.!

Typparameter! (in Java: *Generics*)

`List<T>`

`List<String> s = ...`

- ▶ Klassen und Interfaces in Java können Typparameter haben
- ▶ wird in Klasse verwendet wie ein konkreter Klassenname
- ▶ erlaubt aber (bei Instanziierung) das Einsetzen beliebiger Klassen
- ▶ Beispiel `Comparator<T>`

8.2 Linear List

Linear Listen

java.uhl.List
X

Fundamentaler Container: *ADT Lineare Liste (Dynamische Sequenz)*

- Semantik: zu jedem Zeitpunkt ist L eine Sequenz $a_1, a_2, \dots, a_n$ von Elementen.

```
1 interface LinearList<Elem, Position extends LinearList.ListPosition> {  
2     Position start();  
3     Position end();  
4     Position next(Position p);  
5     Position previous(Position p);  
6  
7     void insert(Position p, Elem x);  
8     void delete(Position p);  
9     Elem get(Position p);  
10    void set(Position p, Elem x);  
11  
12    /** Represents a position of an element in the list */  
13    interface ListPosition { }  
14  
15    // default methods (siehe unten)  
16 }
```

Linear Listen

Fundamentaler Container: *ADT Lineare Liste (Dynamische Sequenz)*

- Semantik: zu jedem Zeitpunkt ist L eine Sequenz $a_1, a_2, \dots, a_n$ von Elementen.

```
1 interface LinearList<Elem, Position extends LinearList.ListPosition> {  
2     Position start();  
3     Position end();  
4     Position next(Position p);  
5     Position previous(Position p);  
6  
7     void insert(Position p, Elem x);  
8     void delete(Position p);  
9     Elem get(Position p);  
10    void set(Position p, Elem x);  
11  
12    /** Represents a position of an element in the list */  
13    interface ListPosition { }  
14  
15    // default methods (siehe unten)  
16 }
```

- zweiter Typparameter Position hier nötig, weil die Semantik von Position von Implementierung abhängt
 ↪ dazu gleich mehr!

Linear Listen – Positionen

Für $L = a_1, \dots, a_n$ können wir als Positionen einfach Indices $p \in \mathbb{N}_{\geq 1}$ angeben

► $L.start() := 1$ erstes Element

Linear Listen – Positionen

Für $L = a_1, \dots, a_n$ können wir als Positionen einfach Indices $p \in \mathbb{N}_{\geq 1}$ angeben

- ▶ `L.start()` := 1 erstes Element
- ▶ `L.end()` := $n + 1$ *hinter* letztem Element

Linear Listen – Positionen

Für $L = a_1, \dots, a_n$ können wir als Positionen einfach Indices $p \in \mathbb{N}_{\geq 1}$ angeben

- ▶ $L.start() := 1$ erstes Element
- ▶ $L.end() := n + 1$ *hinter* letztem Element
- ▶ $L.next(p) := \begin{cases} p + 1 & \text{falls } p \neq L.end(), \\ \text{Exception} & \text{sonst.} \end{cases}$
- ▶ $L.previous(p) := \begin{cases} p - 1 & \text{falls } p \neq L.start(), \\ \text{Exception} & \text{sonst.} \end{cases}$

(Positionen erscheinen unnötig? $\rightsquigarrow$ siehe unten)

Linear Listen – Operationen

Eigentliche Operationen $L = a_1, a_2, \dots, a_n$ und $p \in [1..n + 1]$:

$$\blacktriangleright L.get(p) := \begin{cases} a_p & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

Linear Listen – Operationen

Eigentliche Operationen $L = a_1, a_2, \dots, a_n$ und $p \in [1..n + 1]$:

$$\blacktriangleright L.get(p) := \begin{cases} a_p & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

$\blacktriangleright L.set(p, x)$ wirft Exception falls $p = L.end()$ und terminiert sonst normal.

$$\text{Danach gilt } L = \begin{cases} a_1, \dots, a_{p-1}, x, a_{p+1}, \dots, a_n & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

Linear Listen – Operationen

Eigentliche Operationen $L = a_1, a_2, \dots, a_n$ und $p \in [1..n + 1]$:

► $L.get(p) := \begin{cases} a_p & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$

► $L.set(p, x)$ wirft Exception falls $p = L.end()$ und terminiert sonst normal.

Danach gilt $L = \begin{cases} a_1, \dots, a_{p-1}, x, a_{p+1}, \dots, a_n & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$

► Nach $L.insert(p, x)$ gilt $L = a_1, \dots, a_{p-1}, \underline{x}, a_p, a_{p+1}, \dots, a_n$.

Beachte: $p = L.end()$ erlaubt! $\rightsquigarrow L.insert(p, x)$ hängt x ans Ende von L an.

Linear Listen – Operationen

Eigentliche Operationen $L = a_1, a_2, \dots, a_n$ und $p \in [1..n + 1]$:

$$\blacktriangleright L.get(p) := \begin{cases} a_p & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

$\blacktriangleright L.set(p, x)$ wirft Exception falls $p = L.end()$ und terminiert sonst normal.

$$\text{Danach gilt } L = \begin{cases} a_1, \dots, a_{p-1}, x, a_{p+1}, \dots, a_n & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

$\blacktriangleright$ Nach $L.insert(p, x)$ gilt $L = a_1, \dots, a_{p-1}, x, a_p, a_{p+1}, \dots, a_n$.

Beachte: $p = L.end()$ erlaubt! $\rightsquigarrow L.insert(p, x)$ hängt x ans Ende von L an.

$\blacktriangleright L.delete(p)$ wirft Exception falls $p = L.end()$ und terminiert sonst normal.

$$\text{Danach gilt } L = \begin{cases} a_1, \dots, a_{p-1}, a_{p+1}, \dots, a_n & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

Wozu Positionen?

Es scheint ganz und gar unnötig kompliziert, dass wir Operationen für Positionen haben.

Wir könnten doch einfach direkt den Index p übergeben; was soll all der Ärger?

Wozu Positionen?

Es scheint ganz und gar unnötig kompliziert, dass wir Operationen für Positionen haben.

Wir könnten doch einfach direkt den Index p übergeben; was soll all der Ärger?

Jein . . . wir haben nämlich eine Subtilität unterschlagen:

*Was passiert bei **insert**(L, p) / **delete**(L, p) mit der Position p ?*

Wozu Positionen?

Es scheint ganz und gar unnötig kompliziert, dass wir Operationen für Positionen haben.

Wir könnten doch einfach direkt den Index p übergeben; was soll all der Ärger?

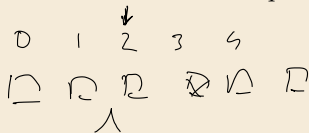
Jein ... wir haben nämlich eine Subtilität unterschlagen:

Was passiert bei **insert**(L, p) / **delete**(L, p) mit der Position p ?

Antwort: Implementierungen von Linearen Listen können Realisierungen wählen!

Index-basierte Positionen

- entspricht obigem Modell einer mathematischen Sequenz



Pointer-basierte Positionen

- entspricht einer verketteten Liste (Index vielleicht gar nicht bekannt)



Wozu Positionen?

Es scheint ganz und gar unnötig kompliziert, dass wir Operationen für Positionen haben.

Wir könnten doch einfach direkt den Index p übergeben; was soll all der Ärger?

Jein . . . wir haben nämlich eine Subtilität unterschlagen:

*Was passiert bei **insert**(L, p) / **delete**(L, p) mit der Position p ?*

Antwort: Implementierungen von Linearen Listen können Realisierungen wählen!

Index-basierte Positionen

- ▶ entspricht obigem Modell einer mathematischen Sequenz
- ▶ $p \in \mathbb{N}_{\geq 1}$ stets das p -te Element vom Anfang auch nach **insert/delete**

Pointer-basierte Positionen

- ▶ entspricht einer verketteten Liste (Index vielleicht gar nicht bekannt)
- ▶ Position verweilt auf „ihrem“ Element auch nach **insert**

Wozu Positionen?

Es scheint ganz und gar unnötig kompliziert, dass wir Operationen für Positionen haben.

Wir könnten doch einfach direkt den Index p übergeben; was soll all der Ärger?

Jein . . . wir haben nämlich eine Subtilität unterschlagen:

*Was passiert bei **insert**(L, p) / **delete**(L, p) mit der Position p ?*

Antwort: Implementierungen von Linearen Listen können Realisierungen wählen!

Index-basierte Positionen

- ▶ entspricht obigem Modell einer mathematischen Sequenz
- ▶ $p \in \mathbb{N}_{\geq 1}$ stets das p -te Element vom Anfang auch nach **insert/delete**

↪ Updates links von p ändern Wert von **get**(p)!

Pointer-basierte Positionen

- ▶ entspricht einer verketteten Liste (Index vielleicht gar nicht bekannt)
- ▶ Position verweilt auf „ihrem“ Element auch nach **insert**

↪ Updates anderswo ändern Wert von **get**(p) **nicht**

Clicker Question

Listen im Collections Framework der Java Runtime Library realisieren unsere Positionen über Iterator-Objekte. Welcher Art von Positionen entsprechen Iteratoren?



- A** Index-basierte Positionen
- B** Pointer-basierte Positionen
- C** Beides
- D** Weder noch
- E** Iter..was?
- F** keine Ahnung



→ *sli.do/cs210*

Clicker Question

Listen im Collections Framework der Java Runtime Library realisieren unsere Positionen über Iterator-Objekte. Welcher Art von Positionen entsprechend Iteratoren?



A ~~Index-basierte Positionen~~ ✓

B Pointer-basierte Positionen ✓

C ~~Beides~~ ✓

D Weder noch ✓

E ~~Iter..was?~~

F ~~keine Ahnung~~



→ *sli.do/cs210*

Abgeleitete Operationen

Einige Hilfsoperationen für Lineare Listen können wir implementieren, ohne zu wissen, welche Listen-Implementierung wir vorliegen haben!

Ultimativer wiederverwendbarer Code!

Abgeleitete Operationen

Einige Hilfsoperationen für Lineare Listen können wir implementieren, ohne zu wissen, welche Listen-Implementierung wir vorliegen haben!

Ultimativer wiederverwendbarer Code!

- ▶ `L.append(x)`: hängt `x` ans Ende von `L` an.

```
1 // public interface LinearList...  
2     default void append(Elem x) {  
3         insert(isEmpty() ? start() : end(), x);  
4     }
```

Abgeleitete Operationen

Einige Hilfsoperationen für Lineare Listen können wir implementieren, ohne zu wissen, welche Listen-Implementierung wir vorliegen haben!

Ultimativer wiederverwendbarer Code!

- `L.append(x)`: hängt `x` ans Ende von `L` an.

```
1 // public interface LinearList...
2     default void append(Elem x) {
3         insert(isEmpty() ? start() : end(), x);
4     }
```

if then else Ausdruck

?

- `L.prepend(x)`: fügt `x` am Anfang (vor allen existierenden Elementen) von `L` an.

```
1     default void prepend(Elem x) { insert(start(), x); }
```

Abgeleitete Operationen

Einige Hilfsoperationen für Lineare Listen können wir implementieren, ohne zu wissen, welche Listen-Implementierung wir vorliegen haben!

Ultimativer wiederverwendbarer Code!

- ▶ `L.append(x)`: hängt `x` ans Ende von `L` an.

```
1 // public interface LinearList...
2     default void append(Elem x) {
3         insert(isEmpty() ? start() : end(), x);
4     }
```

- ▶ `L.prepend(x)`: fügt `x` am Anfang (vor allen existierenden Elementen) von `L` an.

```
1     default void prepend(Elem x) { insert(start(), x); }
```

- ▶ `L.isEmpty()`: liefert `true`, wenn `L` keine Elemente enthält und `false` sonst.

```
1     default boolean isEmpty() { return start().equals(end()); }
```

Abgeleitete Operationen [2]

- ▶ `L.firstOccurrence(x)`: erste Position in `L`, an der `x` in der Liste vorkommt bzw. `L.end()` falls `x` gar nicht in der Liste vorkommt.
- ▶ `L.firstOccurrenceAfter(p,x)`: erste Position in `L` hinter `p` (inklusive `p` selbst), an der `x` in der Liste vorkommt bzw. `L.end()` falls `x` dort nicht in der Liste vorkommt.

```
1  default Position firstOccurrence(Elem x) {  
2      return firstOccurrenceAfter(start(), x);  
3  }  
4  default Position firstOccurrenceAfter(Position p, Elem x) {  
5      for (Position cur = p; !cur.equals(end()); cur = next(cur))  
6          if (get(cur).equals(x)) return cur;  
7      return end();  
8  }  
9  // } end of interface LinearList
```

$\text{for (String } x : l)$

$l : \text{List<String>}$

Abgeleitete Operationen [2]

- ▶ `L.firstOccurrence(x)`: erste Position in `L`, an der `x` in der Liste vorkommt bzw. `L.end()` falls `x` gar nicht in der Liste vorkommt.
- ▶ `L.firstOccurrenceAfter(p,x)`: erste Position in `L` hinter `p` (inklusive `p` selbst), an der `x` in der Liste vorkommt bzw. `L.end()` falls `x` dort nicht in der Liste vorkommt.

```
1  default Position firstOccurrence(Elem x) {  
2      return firstOccurrenceAfter(start(), x);  
3  }  
4  default Position firstOccurrenceAfter(Position p, Elem x) {  
5      for (Position cur = p; !cur.equals(end()); cur = next(cur))  
6          if (get(cur).equals(x)) return cur;  
7      return end();  
8  }  
9  // } end of interface LinearList
```

- ▶ Beachte: Können hier nur die lineare Suche verwenden.